Pizza Hut Sales Analysis
SQL Data Analytics Project

Created by: Jitendra Parihar

Project Overview

Created by Jitendra Parihar

- The primary objective is to analyze Pizza Hut's sales data using SQL to extract actionable business insights
- Focus on identifying trends in customer behavior, product performance, and revenue distribution
- Methodology covers a full spectrum of SQL techniques from beginner to advanced levels
- Transforming raw transactional data into meaningful data-driven stories

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question → Calculate the total revenue generated from pizza sales.

Answer

```
SELECT
ROUND(SUM(orders_details.quantity * pizzas.price), 2) AS total_sales
FROM orders_details
Inner JOIN pizzas
ON pizzas.pizza_id = orders_details.pizza_id;
```

Result Grid

	total_sales
▶	817860.05

Explanation

- This query calculates the total revenue generated from pizza sales.
- The orders_details table contains the quantity of each pizza ordered, while the pizzas table contains the price of each pizza. The INNER JOIN is used to combine both tables using pizza_id, so that each order line has its corresponding price.
- Then, orders_details.quantity * pizzas.price is used to calculate the revenue for each order item. After that, SUM() adds up all these values to get the total revenue from all sales.
- Finally, ROUND(..., 2) is applied to round the result to 2 decimal places for a clean currency format, and AS total_sales gives a meaningful name to the output column.
-

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question → Identify the highest-priced pizza.

Answer

```
SELECT price AS highest_priced_pizza
FROM pizzas
ORDER BY price DESC
LIMIT 1;
```

Result Grid | Filter Rows

	highest_priced_pizza
▶	35.95

Explanation

This query finds the most expensive pizza from the dataset.
The pizzas table contains the price of each pizza, and ORDER BY price DESC is used to sort all pizzas in descending order (highest to lowest).
Then, LIMIT 1 is applied to return only the top result, which represents the pizza with the highest price.
This helps in identifying premium or high-value products in the menu.

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

- Question → Identify the most common pizza size ordered.

Answer

```
SELECT pizzas.size, COUNT(orders_details.order_details_id) AS order_count
FROM pizzas
INNER JOIN orders_details
ON pizzas.pizza_id = orders_details.pizza_id
GROUP BY pizzas.size
ORDER BY order_count DESC;
```

Result Grid | Filter Rows:

size	order_count
L	18526
M	15385
S	14137
XL	544
XXL	28

Explanation

This query determines the most commonly ordered pizza size by analyzing order data. The pizzas table provides information about pizza sizes, while orders_details contains order records.

Using an INNER JOIN on pizza_id, both tables are combined to link each order with its size.

The COUNT() function calculates how many times each size was ordered, and GROUP BY groups the data accordingly.

Finally, ORDER BY order_count DESC sorts the results to show the most popular size at the top.

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question → List the top 5 most ordered pizza types along with their quantities.

Answer

```
SELECT pt.name, SUM(od.quantity) AS total_quantity
FROM orders_details od
INNER JOIN pizzas p
ON od.pizza_id = p.pizza_id
INNER JOIN pizza_types pt
ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY total_quantity DESC
LIMIT 5;
```

Result Grid | Filter Rows:

name	total_quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

Explanation (Single Block):

This query identifies the top 5 most ordered pizza types based on total quantity sold.

The orders_details table provides the quantity of pizzas ordered, while pizzas and pizza_types are used to get the pizza name.

Using INNER JOIN, all three tables are connected to combine quantity and pizza details.

The SUM(od.quantity) calculates the total quantity for each pizza type, and GROUP BY pt.name groups the data by pizza name.

Finally, ORDER BY total_quantity DESC sorts the results from highest to lowest, and LIMIT 5 returns the top 5 pizzas.

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question →Find the total quantity of each pizza category ordered.

Answer

```
SELECT pt.category, SUM(od.quantity) AS total_quantity
FROM orders_details od
INNER JOIN pizzas p
ON od.pizza_id = p.pizza_id
INNER JOIN pizza_types pt
ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category
ORDER BY total_quantity DESC;
```

Result Grid | Filter Rows:

category	total_quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

Explanation

This query calculates the total quantity of pizzas ordered for each category.

The orders_details table provides the quantity, while pizzas and pizza_types are used to identify the category of each pizza.

Using INNER JOIN, all tables are connected through pizza_id and pizza_type_id to combine quantity with category information.

The SUM(od.quantity) function adds up the total pizzas sold for each category, and GROUP BY pt.category groups the data accordingly.

Finally, ORDER BY total_quantity DESC sorts the results to show the most popular category at the top.

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question → Determine the distribution of orders by hour of the day.

Answer

```
SELECT HOUR(order_time) AS order_hour,
COUNT(order_id) AS total_orders
FROM orders
GROUP BY order_hour
ORDER BY order_hour;
```

Result Grid | Filter Rows:

order_hour	total_orders
9	1
10	8
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28

Explanation

This query analyzes how orders are distributed throughout the day based on time.

The orders table contains the time when each order was placed, and the HOUR(order_time) function extracts the hour (0–23) from the order time.

The COUNT(order_id) function is used to calculate the total number of orders placed in each hour.

Using GROUP BY order_hour, the data is grouped for each hour, and ORDER BY order_hour sorts the results in chronological order.

This helps in identifying peak hours when the maximum number of orders are placed.

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question →Join relevant tables to find the category-wise distribution of pizzas.

Answer

```
SELECT pt.category, COUNT(p.pizza_id) AS total_pizzas
FROM pizzas p
INNER JOIN pizza_types pt
ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category
ORDER BY total_pizzas DESC;
```

Result Grid | Filter Rows:

category	total_pizzas
Veggie	27
Classic	26
Supreme	25
Chicken	18

Explanation

What this query does:

- JOINpizzas table with pizza_types using pizza_type_id
- GROUP BYpt.category → groups pizzas by category (e.g., Veg, Non-Veg, Classic, etc.)
- COUNTCOUNT(p.pizza_id) → counts how many pizzas exist in each category
- ORDER BYSorts categories by highest number of pizzas

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question → Group the orders by date and calculate the average number of pizzas ordered per day.

Answer

```
SELECT AVG(daily_total) AS avg_pizzas_per_day
FROM ( SELECT o.order_date, SUM(od.quantity) AS daily_total
FROM orders o
INNER JOIN orders_details od
ON o.order_id = od.order_id
GROUP BY o.order_date ) AS daily_orders;
```

Result Grid | Filter Rows:

	avg_pizzas_per_day
▶	138.4749

Explanation

What this query does:

- JOIN
- Joins the orders table with the orders_details table using order_id
- GROUP BY
- Groups the data by o.order_date → calculates results for each day
- SUM
- SUM(od.quantity) → calculates total pizzas ordered per day (daily_total)
- SUBQUERY
- The inner query calculates total pizzas for each day
- AVG
- The outer query uses AVG(daily_total) to calculate the average number of pizzas ordered per day

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question → Determine the top 3 most ordered pizza types based on revenue.

Answer

```
SELECT pt.name,
SUM(od.quantity * p.price) AS total_revenue
FROM orders_details od
INNER JOIN pizzas p
ON od.pizza_id = p.pizza_id
INNER JOIN pizza_types pt
ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY total_revenue DESC
LIMIT 3;
```

Result Grid | Filter Rows:

name	total_revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5

What this query does:

- JOIN
- Joins orders_details with pizzas using pizza_id, and then joins with pizza_types using pizza_type_id
- GROUP BY
- Groups the data by pt.name → each pizza type
- REVENUE CALCULATION
- od.quantity * p.price → calculates revenue for each order
- SUM(od.quantity * p.price) → total revenue per pizza type
- ORDER BY
- Sorts the result in descending order of total revenue
- LIMIT
- LIMIT 3 → returns top 3 pizza types with highest revenue

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question → Calculate the percentage contribution of each pizza type to total revenue.

Answer

```
SELECT pt.name,
ROUND(SUM(od.quantity * p.price), 2) AS revenue,
ROUND(
  SUM(od.quantity * p.price) * 100 /
  SUM(SUM(od.quantity * p.price)) OVER (), 2
) AS percentage_contribution
FROM orders_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY percentage_contribution DESC;
```

Result Grid | Filter Rows: | Export: | Wrap

name	revenue	percentage_contribution
The Thai Chicken Pizza	43434.25	5.31
The Barbecue Chicken Pizza	42768	5.23
The California Chicken Pizza	41409.5	5.06
The Classic Deluxe Pizza	38180.5	4.67
The Spicy Italian Pizza	34831.25	4.26
The Southwest Chicken Pizza	34705.75	4.24
The Italian Supreme Pizza	33476.75	4.09
The Hawaiian Pizza	32273.25	3.95
The Four Cheese Pizza	32265.7	3.95
The Sicilian Pizza	30940.5	3.78
The Pepperoni Pizza	30161.75	3.69
The Greek Pizza	28454.1	3.48
The Mexicana Pizza	26780.75	3.27
The Five Cheese Pizza	26066.5	3.19
The Pepper Salami Pizza	25529	3.12

What this query does:

- JOIN
- Joins orders_details with pizzas using pizza_id, and then joins with pizza_types using pizza_type_id
- GROUP BY
- Groups the data by pt.name → each pizza type
- REVENUE CALCULATION
- SUM(od.quantity * p.price) → calculates total revenue for each pizza type
- ROUND(..., 2) → rounds revenue to 2 decimal places
- WINDOW FUNCTION (TOTAL REVENUE)
- SUM(SUM(od.quantity * p.price)) OVER () → calculates overall total revenue across all pizza types
- PERCENTAGE CALCULATION
- (individual revenue * 100 / total revenue) → calculates percentage contribution of each pizza type
- ORDER BY
- Sorts results in descending order of percentage contribution

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question → Analyze the cumulative revenue generated over time.

Answer

```
SELECT
o.order_date,
ROUND(SUM(od.quantity * p.price), 2) AS daily_revenue,
ROUND(
   SUM(SUM(od.quantity * p.price)) OVER (ORDER BY o.order_date),
2) AS cumulative_revenue
FROM orders o
JOIN orders_details od ON o.order_id = od.order_id
JOIN pizzas p ON od.pizza_id = p.pizza_id
GROUP BY o.order_date
ORDER BY o.order_date;
```

Result Grid | Filter Rows:

order_date	daily_revenue	cumulative_revenue
2015-01-01	2713.85	2713.85
2015-01-02	2731.9	5445.75
2015-01-03	2662.4	8108.15
2015-01-04	1755.45	9863.6
2015-01-05	2065.95	11929.55
2015-01-06	2428.95	14358.5
2015-01-07	2202.2	16560.7
2015-01-08	2838.35	19399.05
2015-01-09	2127.35	21526.4
2015-01-10	2463.95	23990.35
2015-01-11	1872.3	25862.65
2015-01-12	1919.05	27781.7

What this query does:

- JOIN
- Joins orders with orders_details using order_id, and then joins with pizzas using pizza_id
- GROUP BY
- Groups the data by o.order_date → calculates results for each day
- DAILY REVENUE
- SUM(od.quantity * p.price) → calculates total revenue per day
- ROUND(..., 2) → rounds it to 2 decimal places
- WINDOW FUNCTION (CUMULATIVE REVENUE)
- SUM(SUM(od.quantity * p.price)) OVER (ORDER BY o.order_date)
- → calculates running total (cumulative revenue) over time based on date order
- ORDER BY
- Sorts the final result by order_date in ascending order

PIZZA HUT SALES ANALYSIS

Created by Jitendra Parihar

Question → Determine the top 3 most ordered pizza types based on revenue for each pizza category.

Answer

```
        SELECT *
FROM (
  SELECT
    pt.category,
    pt.name,
    ROUND(SUM(od.quantity * p.price), 2) AS revenue,
    RANK() OVER (
      PARTITION BY pt.category
      ORDER BY SUM(od.quantity * p.price) DESC
    ) AS rank_in_category
  FROM orders_details od
  JOIN pizzas p ON od.pizza_id = p.pizza_id
  JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
  GROUP BY pt.category, pt.name
) AS ranked_pizzas
WHERE rank_in_category <= 3;
```

Result Grid | Filter Rows: | Export: | Wrap C

category	name	revenue	rank_in_category
Chicken	The Thai Chicken Pizza	43434.25	1
Classic	The Classic Deluxe Pizza	38180.5	1
Supreme	The Spicy Italian Pizza	34831.25	1
Veggie	The Four Cheese Pizza	32265.7	1
Chicken	The Barbecue Chicken Pizza	42768	2
Classic	The Hawaiian Pizza	32273.25	2
Supreme	The Italian Supreme Pizza	33476.75	2
Veggie	The Mexicana Pizza	26780.75	2
Chicken	The California Chicken Pizza	41409.5	3
Classic	The Pepperoni Pizza	30161.75	3
Supreme	The Sicilian Pizza	30940.5	3
Veggie	The Five Cheese Pizza	26066.5	3

What this query does:

- JOIN
- Joins orders_details, pizzas, and pizza_types tables to combine order quantity, price, and pizza type details
- REVENUE CALCULATION
- SUM(od.quantity * p.price) → calculates total revenue for each pizza type
- ROUND(..., 2) → rounds revenue to 2 decimal places
- GROUP BY
- Groups data by pt.category and pt.name → each pizza type within each category
- WINDOW FUNCTION (RANKING PER CATEGORY)
- RANK() OVER (PARTITION BY pt.category ORDER BY revenue DESC)
- → assigns rank to pizza types within each category separately based on revenue
- PARTITION BY
- Splits data into groups (categories like Veg, Non-Veg, etc.) before ranking
- SUBQUERY
- Inner query calculates revenue + rank
- Outer query filters only top 3 per category
- FILTERING
- WHERE rank_in_category <= 3 → keeps top 3 highest revenue pizza types in each category

SQL Concepts Covered

Beginner Level Queries

- Total Revenue Calculation: SUM(quantity * price) to determine the overall financial success
- Orders Distribution by Hour: Extracting hour from order_time to identify peak service periods
- Average Pizzas Per Day: Using subqueries to find the daily volume and calculating the mean
- Each query includes a clear explanation of its business value for operational planning

Intermediate Level Queries

- Top 5 Most Ordered Pizzas: Identifying high-volume items to optimize inventory and marketing
- Category-wise Pizza Distribution: Breaking down the menu into Classic, Veggie, Supreme, and Chicken
- Quantity by Category: Understanding which pizza styles drive the most individual unit sales
- Step-by-step logic explaining how multiple JOINs connect sales to pizza names

Advanced Level Queries

- Revenue Contribution Percentage: Calculating how much each pizza category contributes to total sales
- Cumulative Revenue Over Time: Tracking growth trends throughout the fiscal period using window functions
- Top 3 Pizzas by Revenue: Filtering the most profitable items to prioritize premium offerings

Created by Jitendra Parihar

Key Insights

- Most Popular Pizza: The 'Classic' category and specific top-selling names dominate the volume
- Highest Revenue Generator: Premium pizzas often drive higher margins despite lower individual counts
- Peak Order Times: Lunch (12 PM - 1 PM) and Dinner (5 PM - 6 PM) are the busiest windows
- Customer Preference: Large size pizzas tend to be the preferred choice for most orders

Conclusion

- SQL serves as a powerful bridge between raw business data and strategic decision-making
- Data-driven insights help Pizza Hut optimize its menu, staffing, and marketing strategies
- Continuous analysis is essential for maintaining a competitive edge in the food industry
-

Learning Outcome

- Mastery of SQL: Building a strong foundation in complex query writing and database management
- Data-Driven Thinking: Learning how to ask the right business questions from raw datasets
- Problem Solving: Ability to solve real-world retail challenges through technical analysis
- Portfolio Ready: Developing a professional analytics project suitable for industry presentation

Thank You

Thank You for your time and attention!

Questions and Discussion

Presented by: Jitendra Parihar

THANK YOU!